PROMPT PARA CURSOR — Desarrollo de “CodeQuest”

Plataforma Digital Inclusiva para el Fortalecimiento del Pensamiento Lógico en Niños, Niñas y Adolescentes con TEA

> *Copia y pega este documento completo como prompt inicial en Cursor (o pégalo en el chat de Composer/Agent). Está redactado para que la IA entienda el contexto, los objetivos, los usuarios y la arquitectura esperada, y pueda generar el proyecto completo paso a paso.*

---

1. CONTEXTO DEL PROYECTO

Eres un desarrollador full-stack encargado de construir **CodeQuest**, una plataforma web educativa inclusiva basada en **programación por bloques**, diseñada para **fortalecer el pensamiento lógico en niños, niñas y adolescentes con Trastorno del Espectro Autista (TEA)**.

El proyecto nace de una investigación académica (Ingeniería en TIC) con enfoque mixto (cualitativo + cuantitativo), por lo que la plataforma debe además **registrar datos de uso y desempeño** (aciertos, intentos, tiempos de resolución, progreso) para que puedan analizarse posteriormente como evidencia del impacto de la herramienta.

Objetivo general: Desarrollar una plataforma digital educativa inclusiva basada en programación por bloques para fortalecer el pensamiento lógico en NNA (niños, niñas y adolescentes) con TEA.

Objetivos específicos que la plataforma debe soportar: 1. Permitir identificar/registrar las necesidades de pensamiento lógico de cada usuario (mediante una prueba diagnóstica inicial). 2. Ofrecer una plataforma web educativa **adaptativa** basada en bloques de programación. 3. Implementar actividades de programación por bloques **adaptadas** a las características de los usuarios con TEA (dificultad progresiva y ajustable).

---

2. PRINCIPIOS DE DISEÑO (NO NEGOCIABLES)

Este es el punto más importante: la audiencia principal son niños con TEA, así que “bonito y llamativo” tiene reglas especiales. Aplica siempre estos principios:

- **Colorido pero controlado**: paleta vibrante y alegre, pero sin combinaciones estridentes ni saturación extrema. Nada de fondos con gradientes psicodélicos ni parpadeos.
- **Sin sobrecarga sensorial**: evita animaciones rápidas, autoplay de sonidos fuertes, transiciones bruscas o múltiples elementos moviéndose a la vez. Las animaciones deben ser suaves (ease-in-out, 200-400ms) y opcionales/desactivables.
- **Predictibilidad y consistencia**: los mismos elementos (botones, iconos, ubicación del menú) deben verse y comportarse igual en toda la app. Nada de layouts que cambian sorpresivamente.
- **Contraste alto pero agradable**: texto siempre legible (mínimo AA de WCAG), pero usando colores pastel/saturados de fondo, no blanco puro agresivo ni negro puro.
- **Feedback inmediato y claro**: cada acción (arrastrar un bloque, acertar, fallar) debe tener una respuesta visual y textual inmediata, simple y no ambigua.
- **Modo de bajo estímulo**: incluye un interruptor “Modo Calma” que reduce animaciones, colores y sonidos para usuarios que lo necesiten.
- **Tipografía**: redondeada, clara, tamaño grande (mínimo 16px, ideal 18-20px en actividades), buen interlineado. Evitar fuentes decorativas.
- **Iconografía consistente y literal**: iconos claros y realistas, no abstractos ni ambiguos.

Paleta de colores sugerida (llamativa + sensorialmente amigable)

```
Primario:        #4D96FF (azul cielo vibrante, calmante)
Secundario:      #6BCB77 (verde menta, positivo/aciertos)
Acento:          #FFD93D (amarillo cálido, para logros/estrellas)
Alerta suave:    #FF9F5A (naranja suave, NO rojo agresivo para errores)
Fondo claro:     #FAFBFF
Fondo modo calma:#EFF3F8
Texto principal: #2E2E3A
Superficies:     #FFFFFF con sombras suaves (box-shadow difuso, no bordes duros)
```

Usa estos tonos como base y genera variantes (hover, disabled, dark-mode opcional) manteniendo el mismo nivel de saturación entre sí para que nada “grite” más que el resto.

---

3. USUARIOS / ROLES DE LA PLATAFORMA

La plataforma debe tener **3 roles con vistas y flujos completamente distintos**:

3.1. Estudiante (niño, niña o adolescente con TEA)

- Vista simplificada, gamificada, con mínimo texto y máximo apoyo visual.
- Acceso a: perfil con avatar, mapa de niveles/retos, actividades de programación por bloques, sistema de logros/estrellas, modo calma.

3.2. Padre/Madre/Cuidador

- Portal de seguimiento del progreso de su hijo/a (gráficos simples de avance, aciertos, tiempo de uso).
- Foro/espacio de intercambio de experiencias con otras familias.
- Recursos informativos sobre TEA y sobre cómo acompañar el proceso.

3.3. Profesional (docente / psicólogo)

- Panel de monitoreo con trazabilidad detallada por estudiante: aciertos, intentos, tiempos de resolución, secuencia de acciones, evolución histórica.
- Capacidad de exportar reportes (CSV/PDF) para uso clínico o pedagógico.
- Vista comparativa pretest/postest.

(Opcional pero recomendado: un rol Administrador para gestionar usuarios y contenidos de los retos.)

---

4. MÓDULOS FUNCIONALES

4.1. Módulo de Retos (Estudiante) — núcleo de la plataforma

- Editor de **programación por bloques** (usar la librería **Blockly** de Google, es gratuita, madura y pensada exactamente para esto).
- Niveles progresivos: secuenciación → condicionales simples → bucles → lógica combinada.
- Cada reto: instrucción visual simple (con apoyo de íconos/pictogramas, no solo texto), objetivo claro, área de bloques disponibles, área de "ejecución" donde se ve el resultado (ej. un personaje que se mueve en una cuadrícula tipo juego).
- Sistema de intentos ilimitados sin penalización visible negativa (el error se trata como parte del aprendizaje, feedback tipo "casi lo tienes, prueba de nuevo" en vez de "incorrecto").
- Registro automático por actividad de: aciertos, número de intentos, tiempo de resolución, secuencia de bloques usada.

4.2. Motor Adaptativo (el "cerebro" de la plataforma)

Debe ajustar automáticamente la dificultad según el desempeño reciente del usuario, usando como mínimo estas variables: - **% de aciertos** en los últimos N retos. - **Tiempo de resolución** comparado con el tiempo esperado del reto. - **Secuencia de acciones** (número de intentos fallidos antes de acertar).

Lógica sugerida (ajustable): - Si % aciertos > 80% y tiempo bajo → sube un nivel de dificultad. - Si % aciertos < 50% o muchos intentos fallidos repetidos → baja dificultad o repite variante del mismo nivel con más apoyo visual (pistas). - En rango intermedio → mantiene nivel actual.

Implementa esto como un servicio/función independiente (`adaptiveEngine.ts` o similar) para que sea fácil de ajustar y de auditar con fines de investigación.

4.3. Prueba Diagnóstica (Pretest) y Prueba Final (Postest)

- Batería corta de actividades de línea base antes de iniciar el uso regular, para establecer el nivel inicial de pensamiento lógico.
- Misma estructura de prueba disponible al final del periodo de implementación, para comparar resultados (usado por el profesional en su panel).

4.4. Portal de Padres

- Dashboard con gráficas simples (línea de progreso, barras de aciertos por semana).
- Sección "Foro" para que las familias compartan experiencias (CRUD básico de publicaciones y comentarios).
- Sección de recursos/artículos sobre TEA.

4.5. Panel de Profesionales

- Tabla/dashboard por estudiante con métricas detalladas y exportables.
- Gráficos comparativos pretest vs. postest.
- Notas/observaciones que el profesional puede registrar por sesión.

4.6. Sistema de Logros y Gamificación (motivación, no distracción)

- Estrellas o insignias simples por reto completado.
- Mapa de niveles tipo "camino" (como Duolingo) para dar sensación de progreso claro y predecible.

5. STACK TECNOLÓGICO SUGERIDO

Basado en lo mencionado en el proyecto de investigación (Firebase, Visual Studio) y en lo más práctico para que Cursor lo construya rápido:

- **Frontend:** React + TypeScript + Tailwind CSS (permite iterar rápido en el sistema de diseño descrito arriba).
- **Motor de bloques:** Blockly (Google) o Blockly-React.
- **Backend / Base de datos:** Firebase (Firestore + Authentication + Hosting) — coincide con lo ya presupuestado en el proyecto y evita montar servidor propio.
- **Gráficos:** Recharts o Chart.js para los dashboards de padres y profesionales.
- **Autenticación:** Firebase Auth con roles (estudiante / padre / profesional), vinculando cuentas de padre-hijo y profesional-estudiante.
- **Hosting:** Firebase Hosting (ya está contemplado como costo en el proyecto).

Si prefieres evitar Firebase, alternativa: Next.js + PostgreSQL + Prisma + NextAuth — pero Firebase es más rápido para un MVP académico con presupuesto ya asignado a esa herramienta.

6. MODELO DE DATOS (COLECCIONES SUGERIDAS EN FIRESTORE)

```
users
  - id, nombre, rol (estudiante|padre|profesional|admin), email
  - (si es estudiante) padreId, profesionalId, avatar, nivelActual

challenges
  - id, titulo, nivelDificultad, tipoLogica (secuencia|condicional|bucle),
    bloquesDisponibles[], objetivoVisual, tiempoEsperadoSeg

attempts
  - id, estudianteId, challengeId, fechaHora, aciertos (bool),
    numeroIntentos, tiempoResolucionSeg, secuenciaBloquesUsada[]

progress
  - id, estudianteId, nivelActual, estrellasTotales, historialNiveles[]

forumPosts
  - id, autorId, titulo, contenido, fecha, comentarios[]

diagnosticResults
  - id, estudianteId, tipo (pretest|postest), puntaje, fecha, detalle{}
```

7. ACCESIBILIDAD TÉCNICA (checklist para Cursor)

- Cumplir WCAG 2.1 nivel AA como mínimo (contraste, tamaños de foco, navegación por teclado).
- Todo elemento interactivo con `aria-label` claro.
- Nada de contenido que dependa solo del color para transmitir significado (usar también íconos/texto).
- Opción de aumentar tamaño de texto y activar “Modo Calma” (reduce animaciones vía `prefers-reduced-motion` y un toggle manual).
- Sonidos siempre opcionales, con volumen bajo por defecto y botón de silenciar visible.

8. ALCANCE SUGERIDO PARA UN PRIMER MVP (lo que le pides a Cursor que construya primero)

1. Landing page simple explicando la plataforma (para padres/profesionales que llegan por primera vez).
2. Autenticación con 3 roles.
3. Vista de Estudiante: mapa de niveles + un editor de bloques funcional con 3-5 retos de ejemplo (secuenciación básica).
4. Motor adaptativo básico (reglas simples descritas en 4.2).
5. Registro de intentos en Firestore.
6. Dashboard básico de Padre (progreso del hijo).
7. Dashboard básico de Profesional (tabla de intentos + exportar CSV).
8. Aplicar el sistema de diseño (colores, tipografía, modo calma) descrito en la sección 2.

9. INSTRUCCIÓN FINAL PARA CURSOR

> *“Construye este proyecto paso a paso empezando por el sistema de diseño (colores, componentes base en Tailwind: botones, tarjetas, inputs), luego la autenticación con roles, luego el módulo de retos con Blockly, y finalmente los dos dashboards (padres y profesionales). En cada paso, muéstrame el resultado antes de continuar con el siguiente módulo. Prioriza que la interfaz del estudiante sea simple, colorida, predecible y libre de sobrecarga sensorial, siguiendo estrictamente la sección 2 de este documento.”*

Documento generado a partir del proyecto de investigación “CodeQuest” (Universidad Libre Seccional Cúcuta, Ingeniería en TIC).